

BHTTP: a binary HTTP-style protocol

BHTTP carries file requests over one persistent TCP connection using length-prefixed binary frames. This document is enough to write an interoperable server or client. Keywords MUST / MAY follow RFC 2119. All integers are big-endian (network order).

1. Transport and connection model

- TCP. The client opens one connection and sends its request(s) on it; it MUST NOT open a second connection.
- The server MUST keep the connection open after each response and serve further requests until the client closes it. Requests are answered in order.

2. Frame header (fixed, 8 bytes)

Offset	Field	Bits	Meaning
0	Length	24	Payload size in bytes (header excluded). Max 16 777 215
3	Type	8	Frame type (section 3)
4	Flags	8	Type-specific bits; undefined bits MUST be sent as 0 and MUST be ignored
5	Request ID	24	Chosen by the client; every frame of the reply echoes it

Rationale:

- Length 24 bits: 16 MiB per frame is far above any useful chunk, and it matches HTTP/2. Larger bodies are split across DATA frames (bserve uses 64 KiB).
- Type 8 bits: 256 types, so the protocol can grow without a new header.
- Flags 8 bits: room for per-type switches; only END_STREAM is defined.
- Request ID 24 bits: matches replies to requests on a persistent connection and leaves room for multiplexing in a later version.
- The header is 8 bytes, so every field is byte-aligned and the header is word-aligned.

3. Frame types

- 0x01 REQUEST (client to server). Payload: method:u8 (1 = GET) | path_len:u16 | path (UTF-8, starts with /) | header block.
- 0x02 RESPONSE (server to client). Payload: status:u16 (HTTP numbers: 200, 400, 404) | header block. Sent once per request, before any DATA.
- 0x03 DATA (server to client). Payload: raw body bytes. Flag 0x01 = END_STREAM, set on the last DATA frame of a response. A response with no body (400/404) sends one empty DATA frame with END_STREAM.
- Unknown types: a receiver that meets a frame Type it does not know MUST read Length bytes, discard them, and continue. This is the extension point for version 2.

4. Header block

count:u8, then count entries. Each entry starts with a marker byte:

- 0x80 | i (i = 1..10): indexed name from the static table below, then value_len:u16 | value (UTF-8).
- 0x00: literal name: name_len:u8 | name (ASCII, lowercase) | value_len:u16 | value.
- Any other marker is malformed.

Static table: 1 host, 2 user-agent, 3 accept, 4 content-length, 5 content-type, 6 server, 7 date, 8 last-modified, 9 etag, 10 connection.

5. Server behavior

- Path / (or any directory) maps to index.html in that directory. The resolved real path MUST stay inside the root; anything else is treated as not found.
- File exists: RESPONSE 200 (headers server, content-type, content-length), then DATA frames ending with END_STREAM.
- File missing or outside the root: RESPONSE 404 plus an empty END_STREAM DATA frame.
- REQUEST payload malformed (truncated, bad marker, trailing bytes, non-UTF-8, unknown method) while the frame header was intact: RESPONSE 400 plus an empty END_STREAM DATA frame. The stream is still in sync, so the connection stays open.
- A frame whose Length exceeds the server limit (64 KiB for inbound frames): the server sends 400 for that Request ID and MUST close, because it will not read the oversize payload.

- Frames other than REQUEST (including unknown types) are skipped per section 3.
- A connection that ends in the middle of a frame is dropped silently.

6. Client behavior

Send REQUEST, read frames (skipping unknown types) until the RESPONSE and then the DATA frame with END_STREAM, write the body to stdout, and exit 0 for status < 400, 22 for 4xx/5xx, and 1 for network or protocol errors. -v hexdumps every frame sent (>) and received (<).

7. Annotated hexdump (real run: ./bcurl -v localhost:9000/index.html)

Request frame (8 + 48 bytes)

```
00 00 30      Length = 48
01           Type = REQUEST
00           Flags = 0
00 00 01      Request ID = 1
--- payload ---
01           method = GET
00 0b         path_len = 11
2f 69 6e 64 65 78 2e 68 74 6d 6c    "/index.html"
03           header count = 3
81           indexed name 1 = host
00 0e         value_len = 14
6c 6f 63 61 6c 68 6f 73 74 3a 39 30 30 30    "localhost:9000"
82           indexed name 2 = user-agent
00 07         value_len = 7
62 63 75 72 6c 2f 31                "bcurl/1"
83           indexed name 3 = accept
00 03         value_len = 3
2a 2f 2a                             "*/*"

```

Response frame (8 + 31 bytes)

```
00 00 1f      Length = 31
02           Type = RESPONSE
00           Flags = 0
00 00 01      Request ID = 1
--- payload ---
00 c8         status = 200
03           header count = 3
86           indexed name 6 = server
00 08         value_len = 8
62 73 65 72 76 65 2f 31                "bserve/1"
85           indexed name 5 = content-type
00 09         value_len = 9
74 65 78 74 2f 68 74 6d 6c            "text/html"
84           indexed name 4 = content-length
00 02         value_len = 2
31 32                                     "12"

```

Data frame (8 + 12 bytes)

```
00 00 0c      Length = 12
03           Type = DATA
01           Flags = END_STREAM
00 00 01      Request ID = 1
--- payload ---
3c 68 31 3e 68 69 3c 2f 68 31 3e 0a    "<h1>hi</h1>\n"

```
